PYTHON FOR VIDEO GAMES WITH PYGAME · LESSON 6

Menu, pause and game over - game states

Goal: By the end you can organize the game into states (menu, playing, paused, over) with one variable and switch between them with keyboard events.
Before you start:

- Lesson 5 - the game with collisions, score and lives.

To copy the code: when you copy from a PDF, many viewers drop the spaces at the start of each line, and in Python those spaces matter. Every step is also a `.py` file in the same folder as this guide: open it and copy from there.

Step by step

Menu, pause and game over

Right now, when you lose, the window just closes; a real game does not do that. Today's idea is a single variable, state, that says which moment of the game you are in. With it, when you lose you will see a game over screen. You will have a menu to start, and to play again. And you will be able to pause with the P key.

Step 2

Let's go step by step. At the top, next to the font you already had, add a bigger font, big_font, size eighty, for messages. After the Enemy class, add the function draw_centered. It takes a text, a font and a vertical position, and draws it centered left to right. And before the loop, create the variable state with the value playing. That variable will say which moment of the game you are in.

Your `game.py` now looks like this — highlighted lines are new or changed (file `step-1.py`):

```
import random

import pygame

pygame.init()
screen = pygame.display.set_mode((800, 600))
pygame.display.set_caption("Most Wanted UFO Mini")
clock = pygame.time.Clock()

background = pygame.image.load("assets/background_4.png").convert()
player_image = pygame.image.load("assets/player_1.png").convert_alpha()
bullet_image = pygame.image.load("assets/Bullet.gif").convert_alpha()
enemy_image = pygame.image.load("assets/enemy_1.png").convert_alpha()
shoot_sound = pygame.mixer.Sound("assets/shoot.wav")
hit_sound = pygame.mixer.Sound("assets/collision.wav")
lose_sound = pygame.mixer.Sound("assets/loselife.wav")
font = pygame.font.Font(None, 36)
big_font = pygame.font.Font(None, 80)
```

```
class Enemy(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        self.image = enemy_image
        self.rect = self.image.get_rect()
        self.reset()

    def reset(self):
        self.rect.midbottom = (random.randint(20, 780), random.randint(-500, -40))
        self.speed = random.randint(1, 2)

    def update(self):
        self.rect.y += self.speed


def draw_centered(text, font, y):
    image = font.render(text, True, (255, 255, 255))
    screen.blit(image, image.get_rect(center=(400, y)))


player = player_image.get_rect(midbottom=(400, 590))
speed = 6
bullets = []
cooldown = 0
enemies = pygame.sprite.Group(Enemy() for _ in range(8))
score = 0
lives = 3
state = "playing"

running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    keys = pygame.key.get_pressed()
    if keys[pygame.K_LEFT]:
        player.x -= speed
    if keys[pygame.K_RIGHT]:
        player.x += speed
    player.clamp_ip(screen.get_rect())

    if cooldown > 0:
        cooldown -= 1
    if keys[pygame.K_SPACE] and cooldown == 0:
        bullets.append(bullet_image.get_rect(midbottom=player.midtop))
        shoot_sound.play()
        cooldown = 15

    for bullet in bullets:
        bullet.y -= 10
    bullets = [b for b in bullets if b.bottom > 0]
    enemies.update()

    for enemy in enemies:
        hit = enemy.rect.collidelist(bullets)
        if hit != -1:
            bullets.pop(hit)
            enemy.reset()
```

```
            score += 10
            hit_sound.play()
        elif enemy.rect.top > 600 or enemy.rect.colliderect(player):
            enemy.reset()
            lives -= 1
            lose_sound.play()
    if lives <= 0:
        running = False

    screen.blit(background, (0, 0))
    enemies.draw(screen)
    for bullet in bullets:
        screen.blit(bullet_image, bullet)
    screen.blit(player_image, player)
    hud = font.render(f"Score: {score}   Lives: {lives}", True, (255, 255, 255))
    screen.blit(hud, (10, 10))
    pygame.display.flip()
    clock.tick(60)

pygame.quit()
```

Step 3

Now comes the change that looks big. Write if state == "playing", and move under it, four spaces to the right, everything from the arrow keys down to the collisions; events and drawing stay outside. The green lines are the same as before, just shifted. That way, when state stops being playing, none of this moves. First is movement with the arrow keys. Next is the fire rate, also shifted: cooldown goes down every frame, and if it reached zero and you press the space bar, a new bullet comes out, the shot sound plays, and cooldown goes back to fifteen. After that, the bullets move up, the ones that left the top are removed, and the enemy group moves with update. Last, the collisions: just like in the previous lesson, a bullet that hits adds ten points, and an enemy that touches you or gets away takes a life. The only real change is at the end: when lives reach zero, you no longer set running to False; instead, you set state to game_over.

Your `game.py` now looks like this — highlighted lines are new or changed (file `step-2.py`):

```
import random

import pygame

pygame.init()
screen = pygame.display.set_mode((800, 600))
pygame.display.set_caption("Most Wanted UFO Mini")
clock = pygame.time.Clock()

background = pygame.image.load("assets/background_4.png").convert()
player_image = pygame.image.load("assets/player_1.png").convert_alpha()
bullet_image = pygame.image.load("assets/Bullet.gif").convert_alpha()
enemy_image = pygame.image.load("assets/enemy_1.png").convert_alpha()
shoot_sound = pygame.mixer.Sound("assets/shoot.wav")
hit_sound = pygame.mixer.Sound("assets/collision.wav")
lose_sound = pygame.mixer.Sound("assets/loselife.wav")
font = pygame.font.Font(None, 36)
big_font = pygame.font.Font(None, 80)
```

```
class Enemy(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        self.image = enemy_image
        self.rect = self.image.get_rect()
        self.reset()

    def reset(self):
        self.rect.midbottom = (random.randint(20, 780), random.randint(-500, -40))
        self.speed = random.randint(1, 2)

    def update(self):
        self.rect.y += self.speed


def draw_centered(text, font, y):
    image = font.render(text, True, (255, 255, 255))
    screen.blit(image, image.get_rect(center=(400, y)))


player = player_image.get_rect(midbottom=(400, 590))
speed = 6
bullets = []
cooldown = 0
enemies = pygame.sprite.Group(Enemy() for _ in range(8))
score = 0
lives = 3
state = "playing"

running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    if state == "playing":
        keys = pygame.key.get_pressed()
        if keys[pygame.K_LEFT]:
            player.x -= speed
        if keys[pygame.K_RIGHT]:
            player.x += speed
        player.clamp_ip(screen.get_rect())

        if cooldown > 0:
            cooldown -= 1
        if keys[pygame.K_SPACE] and cooldown == 0:
            bullets.append(bullet_image.get_rect(midbottom=player.midtop))
            shoot_sound.play()
            cooldown = 15

        for bullet in bullets:
            bullet.y -= 10
        bullets = [b for b in bullets if b.bottom > 0]
        enemies.update()

        for enemy in enemies:
            hit = enemy.rect.collidelist(bullets)
            if hit != -1:
                bullets.pop(hit)
                enemy.reset()
```

```
                score += 10
                hit_sound.play()
            elif enemy.rect.top > 600 or enemy.rect.colliderect(player):
                enemy.reset()
                lives -= 1
                lose_sound.play()
        if lives <= 0:
            state = "game_over"

    screen.blit(background, (0, 0))
    enemies.draw(screen)
    for bullet in bullets:
        screen.blit(bullet_image, bullet)
    screen.blit(player_image, player)
    hud = font.render(f"Score: {score}   Lives: {lives}", True, (255, 255, 255))
    screen.blit(hud, (10, 10))
    pygame.display.flip()
    clock.tick(60)

pygame.quit()
```

Step 4

Now show it. At the end of the drawing, right before display.flip, add: if state is game_over, use draw_centered to write GAME OVER with the big font, and the score below it.

Your `game.py` now looks like this — highlighted lines are new or changed (file `step-3.py`):

```
import random

import pygame

pygame.init()
screen = pygame.display.set_mode((800, 600))
pygame.display.set_caption("Most Wanted UFO Mini")
clock = pygame.time.Clock()

background = pygame.image.load("assets/background_4.png").convert()
player_image = pygame.image.load("assets/player_1.png").convert_alpha()
bullet_image = pygame.image.load("assets/Bullet.gif").convert_alpha()
enemy_image = pygame.image.load("assets/enemy_1.png").convert_alpha()
shoot_sound = pygame.mixer.Sound("assets/shoot.wav")
hit_sound = pygame.mixer.Sound("assets/collision.wav")
lose_sound = pygame.mixer.Sound("assets/loselife.wav")
font = pygame.font.Font(None, 36)
big_font = pygame.font.Font(None, 80)


class Enemy(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        self.image = enemy_image
        self.rect = self.image.get_rect()
        self.reset()

    def reset(self):
        self.rect.midbottom = (random.randint(20, 780), random.randint(-500, -40))
        self.speed = random.randint(1, 2)
```

```
    def update(self):
        self.rect.y += self.speed


def draw_centered(text, font, y):
    image = font.render(text, True, (255, 255, 255))
    screen.blit(image, image.get_rect(center=(400, y)))


player = player_image.get_rect(midbottom=(400, 590))
speed = 6
bullets = []
cooldown = 0
enemies = pygame.sprite.Group(Enemy() for _ in range(8))
score = 0
lives = 3
state = "playing"

running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    if state == "playing":
        keys = pygame.key.get_pressed()
        if keys[pygame.K_LEFT]:
            player.x -= speed
        if keys[pygame.K_RIGHT]:
            player.x += speed
        player.clamp_ip(screen.get_rect())

        if cooldown > 0:
            cooldown -= 1
        if keys[pygame.K_SPACE] and cooldown == 0:
            bullets.append(bullet_image.get_rect(midbottom=player.midtop))
            shoot_sound.play()
            cooldown = 15

        for bullet in bullets:
            bullet.y -= 10
        bullets = [b for b in bullets if b.bottom > 0]
        enemies.update()

        for enemy in enemies:
            hit = enemy.rect.collidelist(bullets)
            if hit != -1:
                bullets.pop(hit)
                enemy.reset()
                score += 10
                hit_sound.play()
            elif enemy.rect.top > 600 or enemy.rect.colliderect(player):
                enemy.reset()
                lives -= 1
                lose_sound.play()
        if lives <= 0:
            state = "game_over"

    screen.blit(background, (0, 0))
```

```
    enemies.draw(screen)
    for bullet in bullets:
        screen.blit(bullet_image, bullet)
    screen.blit(player_image, player)
    hud = font.render(f"Score: {score}   Lives: {lives}", True, (255, 255, 255))
    screen.blit(hud, (10, 10))
    if state == "game_over":
        draw_centered("GAME OVER", big_font, 250)
        draw_centered(f"Score: {score}", font, 340)
    pygame.display.flip()
    clock.tick(60)

pygame.quit()
```

Step 5

Run the game, and this time do not touch anything. The enemies get through, and you lose your lives one by one. When you reach zero, GAME OVER appears: the loop keeps running, but nothing moves, because state is not playing.

Try it:

```
python game.py
```

Step 6

Now the menu. Change the starting value of state: instead of playing, it is now menu. That way, when the game opens, nothing moves yet, because the logic only runs when state is playing. In the events, add a new case: KEYDOWN, which arrives only once each time you press a key. If it is Enter and you are not playing or paused, a state you will add at the end, a new game starts from scratch. So only from the menu or the game over screen: state becomes playing, the UFO goes back to the bottom center, zero points, three lives, no bullets, and every enemy goes back to the top with reset. If you forget something, the new game carries data over from the old one.

Your `game.py` now looks like this — highlighted lines are new or changed (file `step-4.py`):

```
import random

import pygame

pygame.init()
screen = pygame.display.set_mode((800, 600))
pygame.display.set_caption("Most Wanted UFO Mini")
clock = pygame.time.Clock()

background = pygame.image.load("assets/background_4.png").convert()
player_image = pygame.image.load("assets/player_1.png").convert_alpha()
bullet_image = pygame.image.load("assets/Bullet.gif").convert_alpha()
enemy_image = pygame.image.load("assets/enemy_1.png").convert_alpha()
shoot_sound = pygame.mixer.Sound("assets/shoot.wav")
hit_sound = pygame.mixer.Sound("assets/collision.wav")
lose_sound = pygame.mixer.Sound("assets/loselife.wav")
font = pygame.font.Font(None, 36)
big_font = pygame.font.Font(None, 80)
```

```
class Enemy(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        self.image = enemy_image
        self.rect = self.image.get_rect()
        self.reset()

    def reset(self):
        self.rect.midbottom = (random.randint(20, 780), random.randint(-500, -40))
        self.speed = random.randint(1, 2)

    def update(self):
        self.rect.y += self.speed


def draw_centered(text, font, y):
    image = font.render(text, True, (255, 255, 255))
    screen.blit(image, image.get_rect(center=(400, y)))


player = player_image.get_rect(midbottom=(400, 590))
speed = 6
bullets = []
cooldown = 0
enemies = pygame.sprite.Group(Enemy() for _ in range(8))
score = 0
lives = 3
state = "menu"

running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
        elif event.type == pygame.KEYDOWN:
            if event.key == pygame.K_RETURN and state not in ("playing", "paused"):
                state = "playing"
                player.midbottom = (400, 590)
                score = 0
                lives = 3
                bullets = []
                for enemy in enemies:
                    enemy.reset()

    if state == "playing":
        keys = pygame.key.get_pressed()
        if keys[pygame.K_LEFT]:
            player.x -= speed
        if keys[pygame.K_RIGHT]:
            player.x += speed
        player.clamp_ip(screen.get_rect())

        if cooldown > 0:
            cooldown -= 1
        if keys[pygame.K_SPACE] and cooldown == 0:
            bullets.append(bullet_image.get_rect(midbottom=player.midtop))
            shoot_sound.play()
            cooldown = 15

        for bullet in bullets:
```

```
            bullet.y -= 10
        bullets = [b for b in bullets if b.bottom > 0]
        enemies.update()

        for enemy in enemies:
            hit = enemy.rect.collidelist(bullets)
            if hit != -1:
                bullets.pop(hit)
                enemy.reset()
                score += 10
                hit_sound.play()
            elif enemy.rect.top > 600 or enemy.rect.colliderect(player):
                enemy.reset()
                lives -= 1
                lose_sound.play()
        if lives <= 0:
            state = "game_over"

    screen.blit(background, (0, 0))
    enemies.draw(screen)
    for bullet in bullets:
        screen.blit(bullet_image, bullet)
    screen.blit(player_image, player)
    hud = font.render(f"Score: {score}   Lives: {lives}", True, (255, 255, 255))
    screen.blit(hud, (10, 10))
    if state == "game_over":
        draw_centered("GAME OVER", big_font, 250)
        draw_centered(f"Score: {score}", font, 340)
    pygame.display.flip()
    clock.tick(60)

pygame.quit()
```

Step 7

Now the drawing. The background is always drawn. Then, if state is menu, you use draw_centered to write the game's name with the big font and, below it, to press Enter to play. Otherwise, in any other state, you draw the same as before: the enemies, the bullets, the UFO and the score, now four spaces to the right, inside the else. That way, on the game over screen, the game is still visible behind it. Last, the game over screen: next to the score, it now says to press Enter to play again, because from there you can also start another game.

Your `game.py` now looks like this — highlighted lines are new or changed (file `step-5.py`):

```
import random

import pygame

pygame.init()
screen = pygame.display.set_mode((800, 600))
pygame.display.set_caption("Most Wanted UFO Mini")
clock = pygame.time.Clock()

background = pygame.image.load("assets/background_4.png").convert()
player_image = pygame.image.load("assets/player_1.png").convert_alpha()
bullet_image = pygame.image.load("assets/Bullet.gif").convert_alpha()
enemy_image = pygame.image.load("assets/enemy_1.png").convert_alpha()
```

```
shoot_sound = pygame.mixer.Sound("assets/shoot.wav")
hit_sound = pygame.mixer.Sound("assets/collision.wav")
lose_sound = pygame.mixer.Sound("assets/loselife.wav")
font = pygame.font.Font(None, 36)
big_font = pygame.font.Font(None, 80)


class Enemy(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        self.image = enemy_image
        self.rect = self.image.get_rect()
        self.reset()

    def reset(self):
        self.rect.midbottom = (random.randint(20, 780), random.randint(-500, -40))
        self.speed = random.randint(1, 2)

    def update(self):
        self.rect.y += self.speed


def draw_centered(text, font, y):
    image = font.render(text, True, (255, 255, 255))
    screen.blit(image, image.get_rect(center=(400, y)))


player = player_image.get_rect(midbottom=(400, 590))
speed = 6
bullets = []
cooldown = 0
enemies = pygame.sprite.Group(Enemy() for _ in range(8))
score = 0
lives = 3
state = "menu"

running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
        elif event.type == pygame.KEYDOWN:
            if event.key == pygame.K_RETURN and state not in ("playing", "paused"):
                state = "playing"
                player.midbottom = (400, 590)
                score = 0
                lives = 3
                bullets = []
                for enemy in enemies:
                    enemy.reset()

    if state == "playing":
        keys = pygame.key.get_pressed()
        if keys[pygame.K_LEFT]:
            player.x -= speed
        if keys[pygame.K_RIGHT]:
            player.x += speed
        player.clamp_ip(screen.get_rect())

        if cooldown > 0:
```

```
            cooldown -= 1
        if keys[pygame.K_SPACE] and cooldown == 0:
            bullets.append(bullet_image.get_rect(midbottom=player.midtop))
            shoot_sound.play()
            cooldown = 15

        for bullet in bullets:
            bullet.y -= 10
        bullets = [b for b in bullets if b.bottom > 0]
        enemies.update()

        for enemy in enemies:
            hit = enemy.rect.collidelist(bullets)
            if hit != -1:
                bullets.pop(hit)
                enemy.reset()
                score += 10
                hit_sound.play()
            elif enemy.rect.top > 600 or enemy.rect.colliderect(player):
                enemy.reset()
                lives -= 1
                lose_sound.play()
        if lives <= 0:
            state = "game_over"

    screen.blit(background, (0, 0))
    if state == "menu":
        draw_centered("Most Wanted UFO Mini", big_font, 250)
        draw_centered("Press ENTER to play", font, 340)
    else:
        enemies.draw(screen)
        for bullet in bullets:
            screen.blit(bullet_image, bullet)
        screen.blit(player_image, player)
        hud = font.render(f"Score: {score}   Lives: {lives}", True, (255, 255, 255))
        screen.blit(hud, (10, 10))
    if state == "game_over":
        draw_centered("GAME OVER", big_font, 250)
        draw_centered(f"Score: {score}  -  ENTER to play again", font, 340)
    pygame.display.flip()
    clock.tick(60)

pygame.quit()
```

Step 8

The pause is just one more state. Below the Enter case, add two cases: if you press the P key while playing, state becomes paused; if you press it while paused, it goes back to playing. They use elif, so only one of them runs per press: with two separate ifs, the second one would take you out of the pause right away. And you use KEYDOWN because it arrives once per press; with get_pressed, the pause would flicker every frame. Finally, after drawing the game, if state is paused, write PAUSED with the big font in the middle of the screen. Since it comes after, it sits on top of the frozen scene.

Your `game.py` now looks like this — highlighted lines are new or changed (file `step-6.py`):

```
import random
```

```
import pygame

pygame.init()
screen = pygame.display.set_mode((800, 600))
pygame.display.set_caption("Most Wanted UFO Mini")
clock = pygame.time.Clock()

background = pygame.image.load("assets/background_4.png").convert()
player_image = pygame.image.load("assets/player_1.png").convert_alpha()
bullet_image = pygame.image.load("assets/Bullet.gif").convert_alpha()
enemy_image = pygame.image.load("assets/enemy_1.png").convert_alpha()
shoot_sound = pygame.mixer.Sound("assets/shoot.wav")
hit_sound = pygame.mixer.Sound("assets/collision.wav")
lose_sound = pygame.mixer.Sound("assets/loselife.wav")
font = pygame.font.Font(None, 36)
big_font = pygame.font.Font(None, 80)


class Enemy(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        self.image = enemy_image
        self.rect = self.image.get_rect()
        self.reset()

    def reset(self):
        self.rect.midbottom = (random.randint(20, 780), random.randint(-500, -40))
        self.speed = random.randint(1, 2)

    def update(self):
        self.rect.y += self.speed


def draw_centered(text, font, y):
    image = font.render(text, True, (255, 255, 255))
    screen.blit(image, image.get_rect(center=(400, y)))


player = player_image.get_rect(midbottom=(400, 590))
speed = 6
bullets = []
cooldown = 0
enemies = pygame.sprite.Group(Enemy() for _ in range(8))
score = 0
lives = 3
state = "menu"

running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
        elif event.type == pygame.KEYDOWN:
            if event.key == pygame.K_RETURN and state not in ("playing", "paused"):
                state = "playing"
                player.midbottom = (400, 590)
                score = 0
                lives = 3
                bullets = []
                for enemy in enemies:
```

```
                    enemy.reset()
            elif event.key == pygame.K_p and state == "playing":
                state = "paused"
            elif event.key == pygame.K_p and state == "paused":
                state = "playing"

    if state == "playing":
        keys = pygame.key.get_pressed()
        if keys[pygame.K_LEFT]:
            player.x -= speed
        if keys[pygame.K_RIGHT]:
            player.x += speed
        player.clamp_ip(screen.get_rect())

        if cooldown > 0:
            cooldown -= 1
        if keys[pygame.K_SPACE] and cooldown == 0:
            bullets.append(bullet_image.get_rect(midbottom=player.midtop))
            shoot_sound.play()
            cooldown = 15

        for bullet in bullets:
            bullet.y -= 10
        bullets = [b for b in bullets if b.bottom > 0]
        enemies.update()

        for enemy in enemies:
            hit = enemy.rect.collidelist(bullets)
            if hit != -1:
                bullets.pop(hit)
                enemy.reset()
                score += 10
                hit_sound.play()
            elif enemy.rect.top > 600 or enemy.rect.colliderect(player):
                enemy.reset()
                lives -= 1
                lose_sound.play()
        if lives <= 0:
            state = "game_over"

    screen.blit(background, (0, 0))
    if state == "menu":
        draw_centered("Most Wanted UFO Mini", big_font, 250)
        draw_centered("Press ENTER to play", font, 340)
    else:
        enemies.draw(screen)
        for bullet in bullets:
            screen.blit(bullet_image, bullet)
        screen.blit(player_image, player)
        hud = font.render(f"Score: {score}   Lives: {lives}", True, (255, 255, 255))
        screen.blit(hud, (10, 10))
    if state == "paused":
        draw_centered("PAUSED", big_font, 300)
    if state == "game_over":
        draw_centered("GAME OVER", big_font, 250)
        draw_centered(f"Score: {score}  -  ENTER to play again", font, 340)
    pygame.display.flip()
    clock.tick(60)

pygame.quit()
```

Step 9

Run the game. It starts in the menu; Enter starts the game. Now press P: everything freezes, and PAUSED appears. Press P again, and the game continues right where it was.

Try it:

```
python game.py
```

What you learned

Let's review. state says which moment of the game you are in. The game logic only runs when state is playing. KEYDOWN is for one-time actions, like starting or pausing; get_pressed, for things you hold, like moving. And the menu, the pause and the game over are just different states. Your exercise: keep the best score, and show it in the menu. In the last lesson the final boss arrives, and you will make your own mod.

- state says which moment of the game you are in
- The logic only runs while playing
- KEYDOWN for one-time actions
- Menu, pause and game over are states

Summary

- A `state` variable (`"menu"`, `"playing"`, `"paused"`, `"game_over"`) says which moment the game is in.
- The logic (move, shoot, collide) goes inside `if state == "playing":`; the loop keeps running in the other states.
- `event.type == pygame.KEYDOWN` arrives once when the key is pressed: use it to start or pause. `get_pressed()` is for keys that are held.
- Starting a game resets `score`, `lives`, `bullets` and the enemies.
- `draw_centered(text, font, y)` centers text with `image.get_rect(center=(400, y))`.
- To indent many lines at once inside an `if`, select them in your editor and press Tab.

Exercise

Start from this lesson's `game.py`.

- Create `best = 0` next to `score`.
- When the game ends (`state = "game_over"`), if `score` is greater than `best`, store it in `best`.
- Show `Best: {best}` in the menu and on the game over screen.
- Bonus: add the `Escape` key to go back to the menu from the pause.

Solution

```
best = 0
# ...
# inside if state == "playing":
    if lives <= 0:
        state = "game_over"
        best = max(best, score)
# when drawing the menu and game over:
draw_centered(f"Best: {best}", font, 400)
# in the events:
elif event.key == pygame.K_ESCAPE and state == "paused":
    state = "menu"
```

Art and sounds: Most Wanted UFO Classic, by Jose Daniel Soto, licensed CC BY 4.0.